Documentation pour l'ontologie DIGIPAL

Eve Grimaud

July 2026

1 Création schéma de l'ontologie

1.1 Présentation des ontologies utilisées et leurs ressources

Pour la réalisation de l'ontologie nous avons utilisé différentes ontologies, les liens pour les différentes ressources se trouve ci-dessous.

- CIDOC CRM : `https://cidoc-crm.org/html/cidoc_crm_v7.1.3.html`
- CRMTex : `https://cidoc-crm.org/extensions/crmtex/html/CRMtex_v2.0.html`
- GOLD : `https://static.vocabs.clariah.nl/docs/gold/2010.html`
- Idiomcat : `https://classicmayan.org/documentations/catalogue.html`
- FRBRoo : `chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://cidoc.ics.forth.gr/docs/frbr_oo/frbr_docs/FRBRoo_V2.0_draft_2013May.pdf`
- Annotation : `https://www.w3.org/ns/oa`
- Archetype

1.2 Présentation du schéma et explication

L'image ci-dessous représente le schéma de l'ontologie, vous pouvez retrouver ce même schéma dans le fichier digipal-ontology.svg présent sur le git.

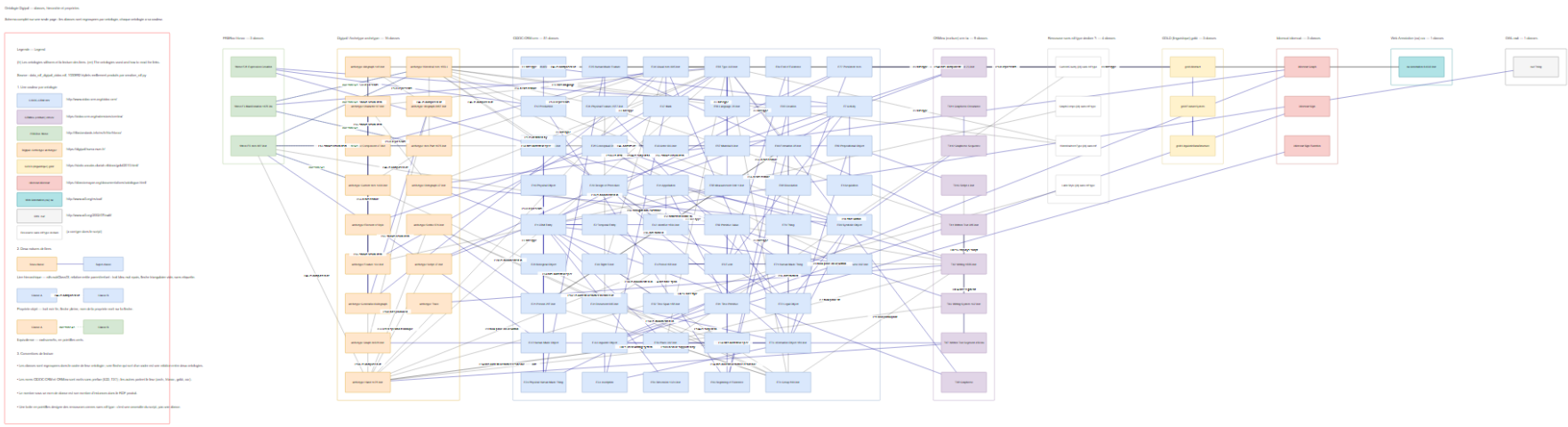


FIGURE 1 – Schéma de l'ontologie

2 Création du fichier RDF via python

2.1 Création d'une base de données avec le dump de la base de données

2.1.1 Création bdd dans postgresl

2.2 Fichier Python

2.2.1 Les bibliothèques utilisées

Le code python utilise différentes bibliothèques comme montre le code si-dessous.

Listing 1 – Listes des imports du contenu dans le fichier python

```
import pandas as pd
import re
from sqlalchemy import create_engine, inspect
from rdflib import Graph, Namespace, RDF, RDFS, OWL, Literal, URIRef, XSD
```

Les différentes bibliothèques utilisées apportent différentes fonctionnalités au code.

- Pandas permet l'analyse de données dans Python, ainsi que une plus puissante et flexible analyses et traitement des données [1].
- Regex permet d'analyser des chaines de caractères en fonction de leurs correspondances à un motif [2].
- sqlalchemy permet au develloper d'utiliser les fonctionnalités de SQL dans Python [3].
- rdflib pour importer les API nécessaires pour créer un fichier RDF [4].

Pour que c'est bibliothèque fonctionne il faut s'assurer qu'elles sont bien installer sur la machine utilisée pour faire tourner le code.

```
pip install pandas rdflib sqlalchemy re
```

2.2.2 L'import des données du dump de la base de données DIGIPAL

La fonction connection permet de connecter le code python avec le dump de la base de données de digipal.

Listing 2 – Fonction pour connecter la base de données au fichier python

```
def connection():
    mydb = create_engine(
    "postgresql+psycopg://postgres:1234@localhost:5432/digipal"
    )
    return mydb
```

Listing 3 – Fonction pour charger les tables en dataframes du dump de la base de données

```
def charger_toutes_tables(engine):
    inspector = inspect(engine)
    tables = inspector.get_table_names(schema="public")
    dataframes = {}
    for table in tables:
        dataframes[table] = pd.read_sql_table(table, engine)
    return dataframes
```

La fonction keep digipal permet de ne garder les tables que nous utilisons pour l'ontologie, c'est à dire les tables qui commence par digipal, ainsi que la table django content type

Listing 4 – Fonction pour garder seulement les tables souhaités

```
def keep_digipal(dico):
    d = {}
    for key in dico.keys():
```

1. Pandas : About pandas https://pandas.pydata.org/about/index.html
2. Documentation Python : Regular expression HOWTOhttps://docs.python.org/fr/3/howto/regex.html
3. SQLAlchemy https://www.sqlalchemy.org/
4. RDFLib https://rdflib.readthedocs.io/en/stable/

```
        if not dico[key].empty:
            if re.match('digipal_', key):
                d[key.replace("digipal_", "")] = dico[key]
            elif re.match('django_content_type', key):
                d['django_content_type'] = dico[key]
    return d
```

L'appel des fonctions et la création du dictionnaire data qui contient

Listing 5 – Appel des fonctions

```
digipal = connection()
dfs = charger_toutes_tables(digipal)
data = keep_digipal(dfs)
```

Ces fonctions créaient des dictionnaires, les données des tables sont contenus dans des dataframes. Les dataframes permettent de garder les même noms des variables de chaque table de la base de données et de les appelé en fonction de ce que l'on souhaite.

Pour vérifier que toutes les tables qui nous souhaitons garder sont bien garder, nous avons créer un fichier txt qui décrit les différentes dataframes des tables de la base de données, avec le type des données et des extraits des données des tables.

Listing 6 – Création du fichier txt qui décrit les données de chacunes des tables gardées

```
txt=""
for table_name, df in data.items():
    txt+=(f"\n——␣{table_name}␣——\n")
    txt+=(f"{df.dtypes}\n")
    txt+=(f"{df.head(2)}\n")

with open("tables_variables.txt", "w", encoding="utf-8") as f:
    f.write(txt)
```

2.2.3 Création du Graphe

Listing 7 – Début de la création du graph

```
    g = Graph()
```

Listing 8 – Namespaces Standards

```
    CRM = Namespace("http://www.cidoc-crm.org/cidoc-crm/")
    FRBRoo = Namespace(
    "http://iflastandards.info/ns/fr/frbr/frbroo/"
    )
    CRMtx = Namespace(
    "https://cidoc-crm.org/extensions/crmtex/html/CRMtex_v2.0.html/"
    )
    Gold = Namespace(
    "https://static.vocabs.clariah.nl/docs/gold/2010.html/"
    )
    Idiomcat = Namespace(
    "https://classicmayan.org/documentations/catalogue.html/"
    )
    ANNO=Namespace("http://www.w3.org/ns/oa#")
    ARCH = Namespace("https://digipal.huma-num.fr/")
    EX = Namespace("http://digipal.eu/ontology#")
    XSD_ = Namespace("http://www.w3.org/2001/XMLSchema#")
```

Listing 9 – ???

```
    g.bind("crm", CRM)
    g.bind("frbroo", FRBRoo)
    g.bind("crm-tx", CRMtx)
    g.bind("gold", Gold)
```

```
g.bind("idiomcat",     Idiomcat)
g.bind("archetype", ARCH)
g.bind("web_annotation", ANNO)
g.bind("ex",       EX)
g.bind("owl",      OWL)
g.bind("xsd",      XSD_)
```

2.2.4 Déclaration de l'ontologie

Listing 10 – ???

```
ontologie = URIRef("http://digipal.eu/ontology")
g.add((ontologie, RDF.type,    OWL.Ontology))
g.add((ontologie, RDFS.label, Literal("Ontologie␣Digipal", lang="en")))
```

Listing 11 – Import OWL des ontologies

```
g.add(
(ontologie, OWL.imports, URIRef
("https://cidoc-crm.org/rdfs/7.1.3/CIDOC_CRM_v7.1.3.rdf")
)
)
g.add(
(ontologie, OWL.imports, URIRef(
"https://cidoc-crm.org/extensions/crmtex/owl/2.0/CRMtex_v2.0.owl"
))
)
```

Pour établir le lien entre les classes enfants et parents, et construire une ontologie, il faut déclarer le lien de parenté une par une, en suivant le modèle ci-dessous.

2.2.5 Déclaration du lien parents/enfants

Listing 12 – Déclaration des classes enfants et parents

```
g.add((Classe.Enfant, RDFS.subClassOf, Classe.Parent))
```

De manière appliqué cela donne :

Listing 13 – Exemple déclaration des classes enfants et parents

```
g.add((FRBRoo.F5_Item, RDFS.subClassOf, CRM.E22_Human_Made_Object))
```

Cela permet de créer un lien entre les différentes ontologies, même on ne télécharge pas leurs ontologies avec les API, comme pour CIDOC CRM et CRMtex.

2.2.6 Déclaration des entités en fonction des variables de la base de données

Création de l'url des entités Fonction URIRef

Boucle for pour les différents dataframes Pour avoir tous les individus de chacune des tables, la boucle for est réalisé sur l'iterrows(), et on ne garde que le deuxième élément

Boucle if pour vérifier que les variables ne sont pas vide pour l'individu en question Les boucles if sont pour ne pas avoir de balises restées vides.

2.2.7 Création du fichier RDF

Listing 14 – Création du fichier rdf

```
g.serialize(destination="data_rdf_digipal_cidoc.rdf", format="xml")
print(f"Fichier␣RDF␣genere␣-␣{len(g)}␣triplets")
```

2.2.8 Lien ente les tables de la base de données et les entités de l'ontologie

Pour la plupart des entités sont les mises dans une seule type RDF, le tableau ci-dessous décrit les tables de la base de données en fonction des types rdf et les propriétés utilisées, si elles ne sont pas juste une table-une entité de manière simple.

Table de la base	Entite choisie	Proprietes utilisees	Notes
digipal_allograph	**archetype :Allograph** Allograph_{id} *148 instances*	*Objet :* crm:P138_represents *Donnees :* ex:default, ex:hidden, ex:illustration, rdfs:label	—
digipal_allographcomponent	*table de jonction* (aucune ressource propre)	*Objet :* crm:P65_shows_visual_item	Aucun rdf:type : la boucle ne cree que des relations entre ressources definies ailleurs.
digipal_annotation	**oa :Annotation** Annotation_{id} *64030 instances*	*Objet :* crm:P138_represents, crm:P148_has_component, crm:P2_has_type *Donnees :* ex:cutout, ex:display_note, ex:geo_json, ex:holes, ex:illustration_ductus, ex:internal_note, ex:rotation, ex:vector_id, rdfs:label	— References non resolues vers Status_{id} : 6702 au total, 6702 au format flottant (URI en _13.0 alors que la ressource existe sous _13 : ajouter int() ou charger la colonne en Int64). — Commentaire du code : « traiter status comme un E55_Type »
digipal_appearance	**archetype :Script** Appearance_{id} *37 instances*	*Donnees :* ex:description, ex:sort_order, ex:text, rdfs:label	—
digipal_archive	**crm :E73_Information_Object** HistoricalItemArchive_{id} *180 instances*	*Objet :* crm:P50_has_current_keeper *Donnees :* ex:dubitable, rdfs:label	— Le type de la clé est changer avec un int(), comme la clé interne de Place est de type int(), alors que la clé étrangère place_id est de type float. — Table lue depuis la jointure archive + django_content_type. Ligne 302 : le gardien n'est ajoute que si content_type_id = 43, il n'existe pas d'autre type, si jamais il venait à être créer, un message serait affiché pour prévenir qu'il faudrait rajouter des lignes de code pour pouvoir le prendre en compte.
digipal_category	**crm :E55_Type** Category_{id} *291 instances*	*Objet :* crm:P2_has_type *Donnees :* ex:sort_order, rdfs:label	—
digipal_character	**archetype :Character** Character_{id} *67 instances*	*Objet :* crm:P138_represents, crm:P2_has_type *Donnees :* ex:unicode_point, rdfs:label	—
digipal_character_components	*table de jonction* (aucune ressource propre)	*Objet :* crm:P65_shows_visual_item	Aucun rdf:type : la boucle ne cree que des relations entre ressources definies ailleurs.
digipal_characterform	**crm :E55_Type** FormCharacter_{id} *3 instances*	*Donnees :* rdfs:label	—
digipal_component	**archetype :Component** Component_{id} *47 instances*	*Donnees :* rdfs:label	—
digipal_component_features	*table de jonction* (aucune ressource propre)	*Objet :* crm:P56_bears_feature	Aucun rdf:type : la boucle ne cree que des relations entre ressources definies ailleurs.

suite page suivante

suite du tableau de correspondance

Table de la base	Entite choisie	Proprietes utilisees	Notes
digipal_county	**crm :E53_Place** County_{id} *67 instances*	*Donnees :* rdfs:label	—
digipal_currentitem	**archetype :Current_Item** CurrentItem_{id} *1490 instances*	*Objet :* crm:P50_has_current_keeper *Donnees :* ex:shelfmark, rdfs:label	— int() a été rajouté comme le type de la clé étrangère et de la clé interne pour faire le lien n'était pas le même.
digipal_date	**crm :E4_Period** Date_Period_{id} *158 instances* *(secondaire)* **crm :E52_Time_Span** Date_TimeSpan_{id} *158 instances* *(secondaire)* **crm :E31_Document** Date_Document_{id} *148 instances*	*Objet :* crm:P4_has_time_span, crm:P70i_is_documented_in *Donnees :* ex:additional_band, ex:band, ex:date, ex:evidence, ex:max_weight, ex:min_weight, ex:post_conquest, ex:s_xi, ex:sort_order, ex:weight, rdfs:label	— Plusieurs rdf:type pour cette table : crm:E4_Period, crm:E52_Time_Span, crm:E31_Document (3 ressources distinctes creees par ligne).
digipal_decoration	**crm :E18_Physical_Thing** Decoration_{id} *1325 instances*	*Objet :* crm:P2_has_type, crm:P46_is_composed_of *Donnees :* ex:catalogue_references, ex:colours, ex:decorated, ex:description, ex:illustrated, ex:inks, ex:num_colours, ex:num_inks, ex:style, rdfs:label	—
digipal_description	**crm :E26_Physical_Feature** Description_{id} *2077 instances*	*Objet :* crm:P56_bears_feature, crm:P70_is_documented_in *Donnees :* ex:comments, ex:description, ex:summary, rdfs:label	— int() utiliser pour assurer que le type des clés étrangères et des clés primaires pour faire le lien entre les différentes entités sont du même type.
digipal_feature	**archetype :Feature** Feature_{id} *134 instances*	*Donnees :* rdfs:label	—
digipal_format	**crm :E55_Type** HistoricalItemFormat_{id} *4 instances*	*Objet :* crm:P2_has_type *Donnees :* rdfs:label	—
digipal_graph	**archetype :Graph** Graph_{id} *64028 instances*	*Objet :* crm:P106_is_composed_of, crm:P65_shows_visual_item *Donnees :* rdfs:label	—
digipal_graphcomponent	*table de jonction* (aucune ressource propre)	*Objet :* crm:P65_shows_visual_item	Aucun rdf:type : la boucle ne cree que des relations entre ressources definies ailleurs.

suite page suivante

suite du tableau de correspondance

Table de la base	Entite choisie	Proprietes utilisees	Notes
digipal_graphcomponent_feature	**crm :E26_Physical_Feature** GraphCompo_{id} *4883 instances*	*Objet :* crm:P56_bears_feature	Les ressources creees ici ne reçoivent aucun rdf:type comme cette table n'est utilisé que pour faire un lien entre plusieurs tables
digipal_hair	**crm :E57_Material** Hair_{id} *6 instances*	*Donnees :* rdfs:label	—
digipal_hand	**archetype :Hand** Hand_{id} *1478 instances* *(secondaire)* **crm-tx :TX3_Writing_System** Writing_System_{id} *132 instances* *(secondaire)* **crm-tx :TX13_Script** Script_{id} *4 instances* *(secondaire)* **frbroo :F5_Item** Item_{id} *687 instances*	*Objet :* crm:P108_has_produced, crm:P2_has_type, crm:P33_used_specific_technique, crm:P46_is_composed_of, crm:P65_shows_visual_item, crm:P74_has_current_or_former_residence, crm:P8_took_place_on_or_within, crm-tx:TXP12_has_style, crm-tx:TXP16_employs_script, crm-tx:TXP1_used_writing_system *Donnees :* ex:comments, ex:display_label, ex:display_note, ex:em_title, ex:gloss_only, ex:imitative, ex:internal_note, ex:ker, ex:latin_only, ex:locus, ex:membra_disjecta, ex:num, ex:num_glosses, ex:num_glossing_hands, ex:relevant, ex:scragg, ex:scribble_only, ex:selected_locus, ex:surrogates, rdfs:label	— Plusieurs rdf:type pour cette table : archetype:Hand, crm-tx:TX3_Writing_System, crm-tx:TX13_Script, frbroo:F5_Item (4 ressources distinctes creees par ligne).
digipal_hand_images	*table de jonction* (aucune ressource propre)	*Objet :* crm:P138_represents	Aucun rdf:type : la boucle ne cree que des relations entre ressources definies ailleurs.
digipal_handdescription	**crm :E62_String** HandDescription_{id} *427 instances*	*Objet :* crm:P3_has_note, crm:P70i_is_documented_in *Donnees :* ex:description, rdfs:label	—

suite page suivante

suite du tableau de correspondance

Table de la base	Entite choisie	Proprietes utilisees	Notes
digipal_historicalitem	**archetype :Historical_Item** HistoricalItem_{id} *1553 instances* *(secondaire)* **crm :E42_Identifier** CatalogueNumber_{id} *1504 instances* *(secondaire)* **crm :E56_Language** Language_{id} *20 instances* *(secondaire)* **crm :E92_Spacetime_Volume** SpacetimeVolume_{id} *602 instances*	*Objet :* crm:P196i_is_defined_by, crm:P1_is_identified_by, crm:P2_has_type, crm:P72_has_language *Donnees :* ex:cataloguenumber, ex:legacy_reference, ex:neumed, ex:url, ex:vernacular, rdfs:label	— Plusieurs rdf:type pour cette table : archetype:Historical_Item, crm:E42_Identifier, crm:E56_Language, crm:E92_Spacetime_Volume (4 ressources distinctes creees par ligne). — Reference des ressources sans rdf:type : HistoricalItemType_{id}.
digipal_historicalitem_categories	*Item_categories* (aucune ressource propre)	*Objet :* crm:P2_has_type	— Aucun rdf:type : la boucle ne cree que des relations entre ressources définies ailleurs.
digipal_historicalitemdate	**crm :E52_Time-Span** Writing_{id} *1555 instances*	*Objet :* crm:P65_shows_visual_item, crm:P8_took_place_on_or_within *Donnees :* ex:addition, ex:dubitable, ex:evidence, ex:vernacular, rdfs:label	References vers Date_Period_{id} : 1323 au total, 1323 au format flottant (URI en _13.0 alors que la ressource existe sous _13 : ajouter int() pour que le lien fonctionne entre les deux entité.
digipal_idiograph	**archetype :Idiograph** Idiograph_{id} *9657 instances*	*Objet :* crm:P138_represents *Donnees :* rdfs:label	— Plusieurs liens qui existaient dans la base de données ne sont pas défini dans l'ontologie. »
digipal_image	**crm :E36_Visual_Item** Image_{id} *995 instances*	*Objet :* crm:P104_is_subject_to, crm:P138_represents *Donnees :* ex:caption, ex:custom_label, ex:folio_side, ex:height, ex:iipimage, ex:image, ex:internal_notes, ex:keywords_string, ex:locus, ex:page_boundaries, ex:quire, ex:size, ex:transcription, ex:width, rdfs:label	—
digipal_imageannotationstatus	**crm :E55_Type** Status_{id} *4 instances*	*Donnees :* rdfs:label	—

suite page suivante

suite du tableau de correspondance

Table de la base	Entite choisie	Proprietes utilisees	Notes
digipal_institution	**crm :E74_Group** Institution_{id} *298 instances* *(secondaire)* **crm :E66_Formation / crm :E8_Acquisition** InstiFoundation_{id} *30 instances* *(secondaire)* **crm :E66_Formation** InstiRefoundation_{id} *8 instances* *(secondaire)* **crm :E53_Place** Place_{id} *307 instances*	*Objet :* crm:P14_carried_out_by, crm:P198i_held_or_supported_by, crm:P22_transferred_title_to, crm:P2_has_type, crm:P74_has_current_or_former_residence, crm:P95_has_formed *Donnees :* ex:foundation, ex:refoundation, rdfs:label	— Plusieurs rdf:type pour cette table : crm:E74_Group, crm:E66_Formation, crm:E8_Acquisition, crm:E53_Place (4 ressources distinctes creees par ligne). — La ressource InstiFoundation_{id} recoit plusieurs classes selon les donnees : crm:E66_Formation, crm:E8_Acquisition. — Ligne 385 : quand founder_id est renseigne, la refondation reutilise l'URI InstiFoundation_{id} de la fondation et la type crm:E8_Acquisition (ligne 386). Fondation et refondation se confondent donc dans une meme ressource, qui porte deux classes. Sinon une URI distincte InstiRefoundation_{id} est creee, typee crm:E66_Formation.
digipal_institutiontype	**crm :E55_Type** InstitutionType_{id} *7 instances*	*Donnees :* rdfs:label	—
digipal_itemorigin	*table de jonction* (aucune ressource propre)	*Objet :* crm:P11_had_participant, crm:P7_took_place_at *Donnees :* ex:dubitable	— Aucun rdf:type : la boucle ne cree que des relations entre ressources definies ailleurs. —
digipal_itempart	**archetype :Item_Part** Itempart_{id} *1675 instances*	*Objet :* crm:P46_is_composed_of, owl:sameAs *Donnees :* ex:pagination, rdfs:comment, rdfs:label	— References non resolues vers Item_{id} : 3952 au total, 3952 vers une ressource qui n'est creee nulle part. — References non resolues vers Itempart_{id} : 992 au total, 992 au format flottant (URI en _13.0 alors que la ressource existe sous _13 : ajouter int() ou charger la colonne en Int64).
digipal_itempart_owners	*table de jonction* (aucune ressource propre)	*Objet :* crm:P50_has_current_keeper	Aucun rdf:type : la boucle ne cree que des relations entre ressources definies ailleurs.

suite page suivante

suite du tableau de correspondance

Table de la base	Entite choisie	Proprietes utilisees	Notes
digipal_itempartitem	**frbroo :F3_Manifestation** Manifestation_{id} *1675 instances*	*Objet :* crm:P46_is_composed_of, crm:P92_brought_into_existance, owl:sameAs	— Le lien créer entre différentes tables, pour pouvoir créer un lien entre writing et les différentes entités enfants de F3 Manifestation.
digipal_language	**crm :E56_Language** Language_{id} *20 instances*	*Donnees :* rdfs:label	—
digipal_latinstyle	**crm-tx :TX10_Style** LatinStyle_{id} *23 instances*	*Donnees :* rdfs:label	L'entité choisie pour Latin Style n'est pas forcément la bonne.
digipal_layout	**crm :E18_Physical_Thing** Layout_historicalItem_{id} *1324 instances* *(secondaire)* **crm :E54_Dimension** Dimension_historicalItem_{id} *1324 instances*	*Objet :* crm:P43_has_dimension, crm:P45_consits_of, crm:P46_is_composed_of *Donnees :* ex:bilinear_ruling, ex:columns, ex:frame_height, ex:frame_width, ex:lines, ex:on_top_line, ex:page_height, ex:page_width, ex:tramline_width, rdfs:comment, rdfs:label	— Plusieurs rdf:type pour cette table : crm:E18_Physical_Thing, crm:E54_Dimension (2 ressources distinctes creees par ligne). — References non resolues vers Hair_{id} : 93 au total, 93 au format flottant (URI en _13.0 alors que la ressource existe sous _13 : ajouter int() ou charger la colonne en Int64). — Ligne 514 : crm:P45_consits_of n'existe pas dans CIDOC-CRM, lire crm:P45_consists_of. Par ailleurs ex:columns porte trois champs differents (lignes 500, 504 et 510) : columns, multiple_sheet_rulling et insular_pricking.
digipal_mediapermission	**crm :E30_Right** MediaPermission_{id} *5 instances*	*Donnees :* ex:display_message, ex:permission, rdfs:label	—
digipal_ontograph	**archetype :Ontograph** Ontograph_{id} *37 instances*	*Objet :* crm:P2_has_type *Donnees :* ex:nesting_level, ex:sort_order, rdfs:label	—
digipal_ontographtype	**crm :E55_Type** OntographType_{id} *5 instances*	*Donnees :* rdfs:label	—
digipal_owner	**crm :E39_Actor** Owner_{id} *993 instances*	*Objet :* crm:P74_has_current_or_former_residence, owl:sameAs *Donnees :* ex:annotated, ex:date, ex:dubitable, ex:evidence, ex:rebound, rdfs:label	Lignes 777 a 783 : les owl:sameAs vers la personne, l'institution.
digipal_ownertype	**crm :E55_Type** OwnerType_{id}	rdf:type, rdfs:label	Bien que la table soit appelée Owner Type, il s'agit enfaîte du type de Repository.
digipal_person	**crm :E21_Person** Person_{id} *287 instances*	*Donnees :* rdfs:label	—
digipal_place	**crm :E53_Place** Place_{id} *307 instances*	*Objet :* crm:P2_has_type, crm:P89_falls_within *Donnees :* ex:eastings, ex:northings, rdfs:label	Reference des ressources sans rdf:type : CurrentCounty_{id}.

suite page suivante

suite du tableau de correspondance

Table de la base	**Entite choisie**	**Proprietes utilisees**	**Notes**
`digipal_reference`	**crm :E31_Document** `Reference_{id}` *440 instances*	*Donnees :* `ex:full_reference`, `ex:legacy_reference`, `rdfs:label`	

2.3 Création du fichier OWL

2.3.1 Ouvrir Protégé

Installation de protégé Pour pouvoir créer un fichier OWL à partir du fichier RDF, il faut avoir le logiciel Protégé, en se dirigeant sur le lien ci-dessous.
`https://protege.stanford.edu/` Vous pouvez trouver la documentation necessaire pour l'installation, en fonction de votre système d'exploitation de votre ordinateur sur ce lien, `https://protegeproject.github.io/protege/installation/`

Si protégé est déjà installé lancer le logiciel.

2.3.2 Charger le fichier rdf

Lancer Protege

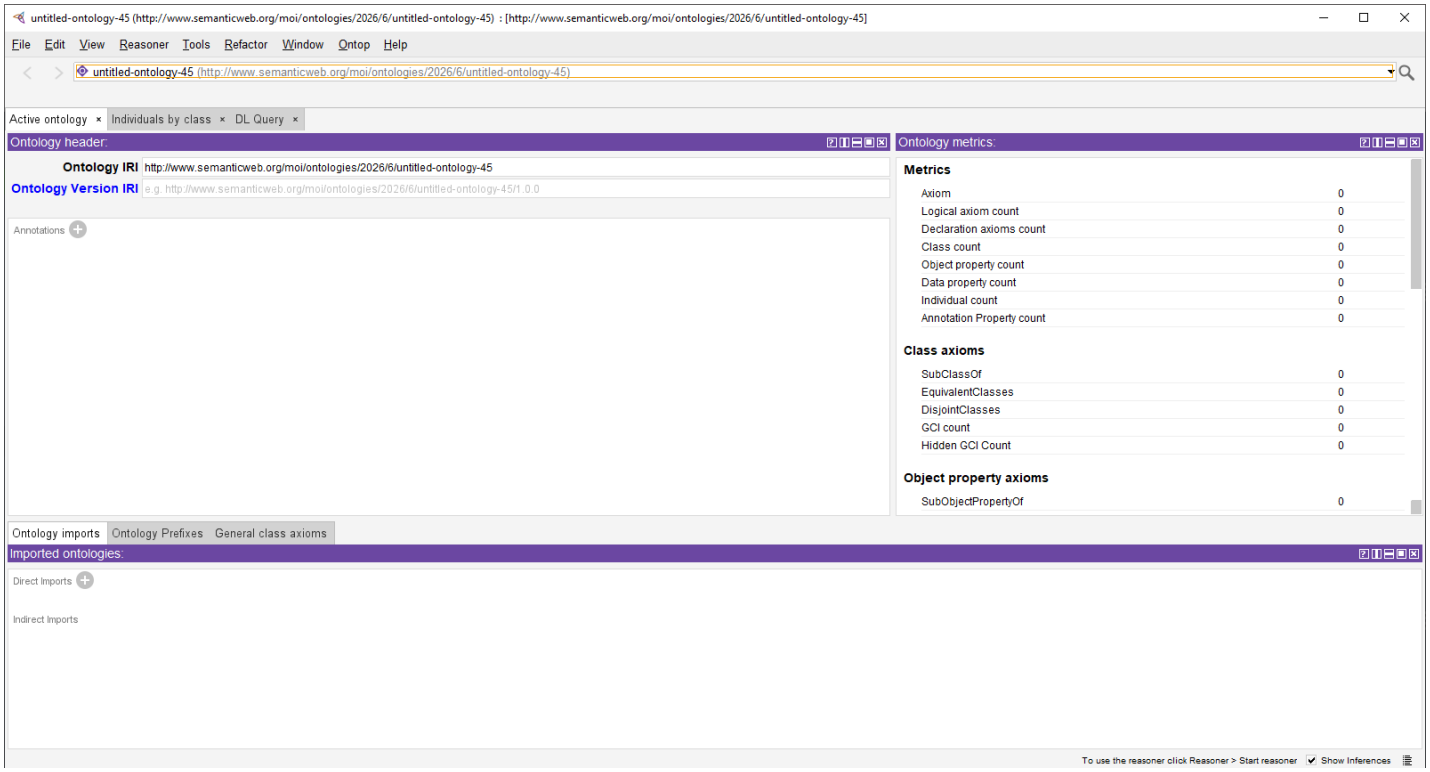


FIGURE 2 – Aperçu du logiciel Protege

Pour ouvrir le fichier rdf créé précédemment, il faut tout d'abord aller dans l'onglet "File" puis sélectionner "Open". Cette fenêtre devrait s'ouvrir, et vous pouvez sélectionner le fichier souhaité et cliquer sur "Ouvrir".

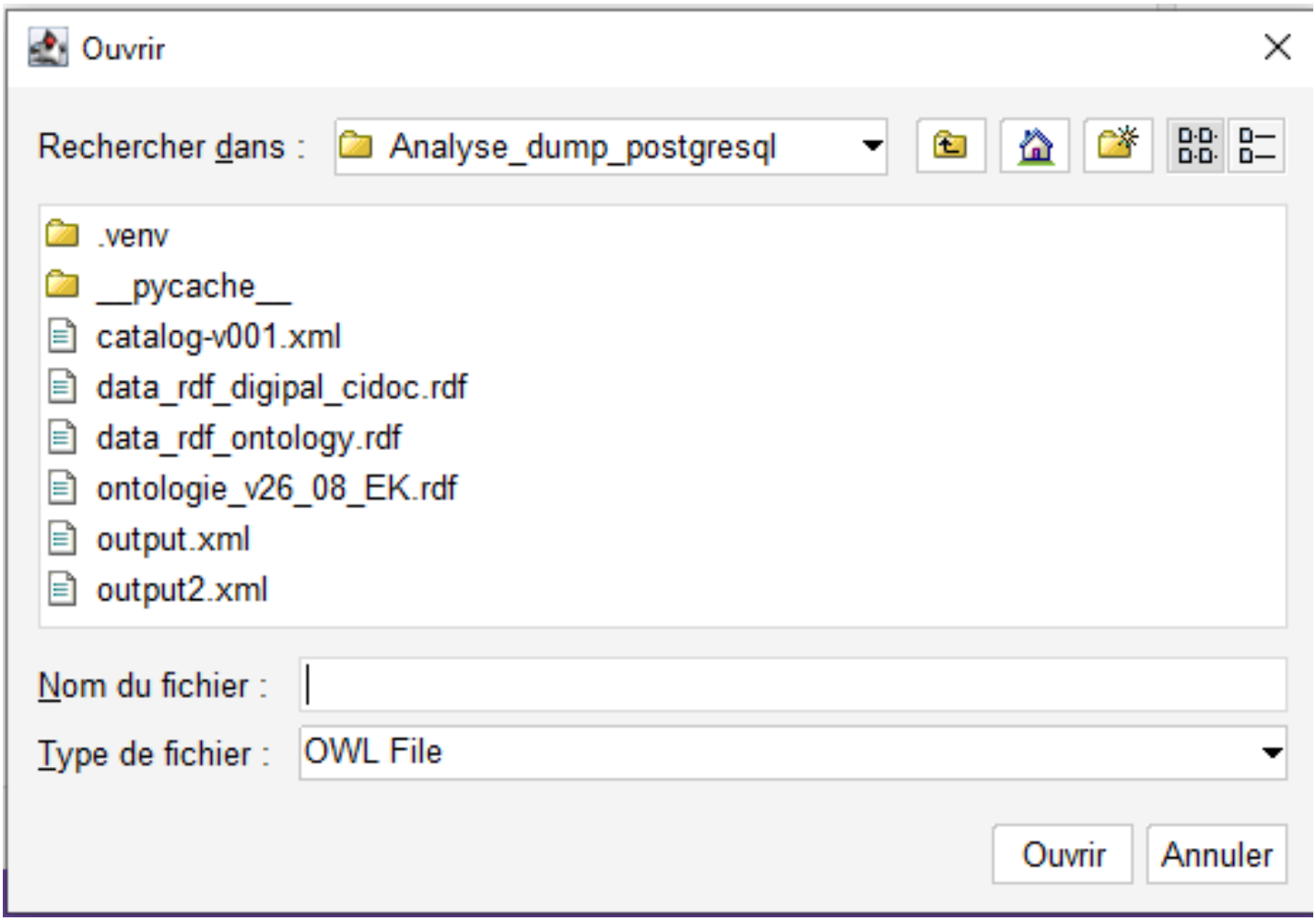


FIGURE 3 – Fenêtre Protege pour sélectionner le fichier

2.3.3 Sauvegarde du fichier sous owl

Après que le fichier rdf a été ouvert, vérifier que tout à bien été, chargé : les entités, les propriétés, et les individus, et enregistrer le fichier en format .owl, en utilisant "Save as" et en sélectionnant le

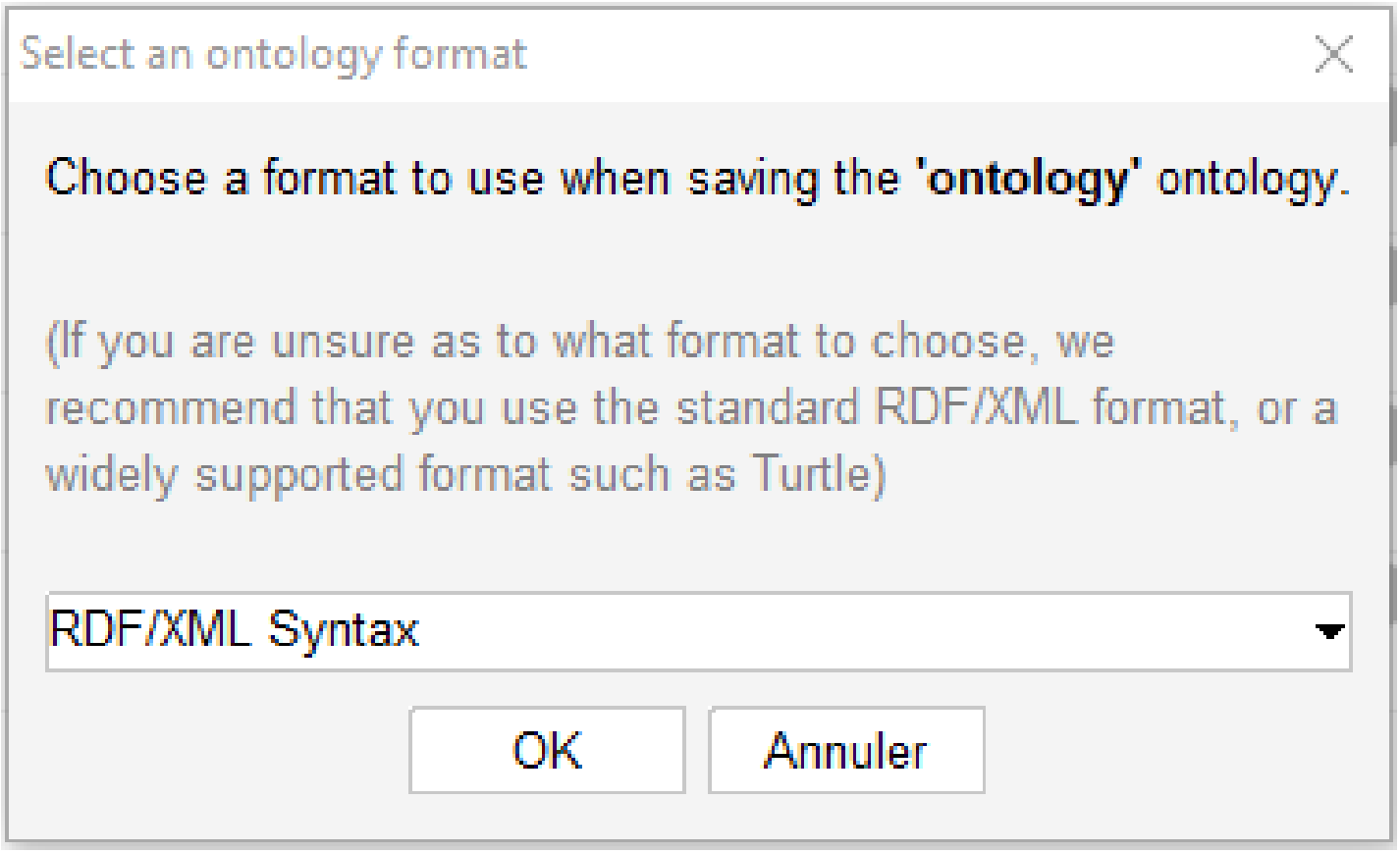


FIGURE 4 – Fenêtre Protege pour sélectionner le fichier